



ENTERPRISE FRONTEND

State Management

A Complete Teaching Resource

Vue.js

Pinia

React

Redux Toolkit

Universal

TanStack Query

State Management · Data Flow · Cache-First Strategy · Optimistic UI · UI Best Practices

Contents

- 01 What is State & Why Do We Need It?**
The GRC platform analogy, three types of state, the golden rule
- 02 Data Flow in Applications**
Unidirectional flow, the newspaper analogy, the universal loop
- 03 Pinia - State Management for Vue.js**
Why Pinia, how it works, state / getters / actions, real patterns
- 04 Redux - State Management for React**
Why Redux, slices, reducers, the Parliament analogy, RTK
- 05 Pinia vs Redux - Key Differences**
Philosophy comparison, when to use which, side-by-side analysis
- 06 Cache-First Data Strategy**
The banking analogy, four-step pattern, TTL, background revalidation
- 07 Optimistic UI Updates**
WhatsApp analogy, pessimistic vs optimistic, rollback, when to use
- 08 UI Best Practices**
Three async states, skeleton screens, smart vs presentational, memoisation
- 09 Full Architecture - How Layers Connect**
Complete end-to-end flow, all layers in one picture
- 10 Recommended Tool Stack**
Pinia, Redux Toolkit, TanStack Query, DevTools
- 11 The 10 Golden Rules**
Non-negotiables every developer on the team must follow

SECTION 01

What is State & Why Do We Need It?

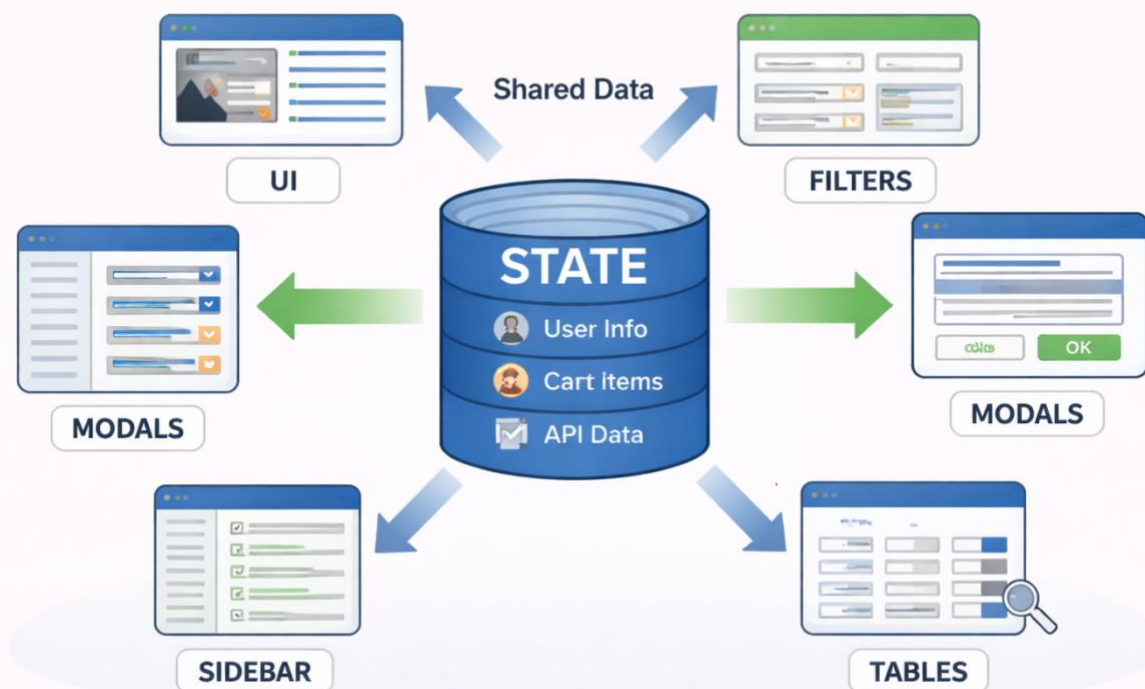
Foundations - The bedrock every developer must understand before writing a single line

The Core Idea

Think of your application as a living, breathing thing. At any moment in time, it has a "condition" - what is currently on screen, who is logged in, what risks are currently open, whether a modal is open. This condition is called state.

Without deliberately managing state, your app becomes chaotic: different parts of the screen show different things, data gets duplicated, and bugs become impossible to track down. State management is the practice of deciding where that data lives, how it changes, and how all parts of your app stay in sync.

State Management Overview



The Restaurant Analogy

Imagine a busy restaurant. The kitchen, the waitstaff, and the billing counter all need to know the same thing - what did Table 7 order, has it been prepared, has it been paid for?

Without a central system: The kitchen notepad says "Pasta, Salad." The waiter's pad says "Pasta, Soup." The billing system says "Pizza, Salad." Result: chaos, contradictions, and angry customers. Each department is maintaining its own version of reality.

With an Order Management System: One central record everyone reads from and writes to. Table 7 is always "Pasta, Salad - Status: Preparing - Paid: No." The kitchen, waiter, and billing counter are all looking at exactly the same information.

This is precisely what a state management store is: a single, authoritative source of truth that every part of your application reads from and updates in a controlled way.

Why This Matters

In a real app, your Header component might show a risk count of 3, while the Risk Dashboard shows 2, because they each fetched and stored data independently. A state store eliminates this class of bug entirely.

State management is needed because every application must **remember something** while it is running.

That something can be:

1. who the logged-in user is
2. what risk was flagged or updated
3. whether a button is loading
4. what page the user is on
5. which data came from the backend
6. whether the data is fresh, changed, saved, failed, or still pending

This remembered information is called **state**.

State management means:

How an application stores, updates, shares, and tracks changing data correctly across the UI and business logic.

Without state, an application would behave like a person with no short-term memory. It could show a screen, but it would not reliably remember what just happened, what the user selected, or what data changed.

Why State Management is Required

State management is required because modern applications are **not static pages**. They are interactive systems.

When a user clicks, types, logs in, filters data, opens a modal, uploads a file, or gets data from an API, the app must respond correctly. For that, it must track changing values over time.

State management is needed mainly for these reasons:

1. To Keep the UI and Data in Sync

1. If the data changes, the UI should change.
2. If the UI changes, it should reflect the real data.

Example:

The Zomato/Swiggy Order Tracking Analogy

Imagine you have ordered biryani from Swiggy. Multiple screens need to stay updated with the same order status.

Without proper sync

- The home screen shows **"Order Placed"**
- The tracking page shows **"Out for Delivery"**
- The notification says **"Delivered"**
- But the delivery boy is still at the restaurant

With proper state management

When the restaurant marks your order as **"Being Prepared,"** every screen updates instantly:

- Home screen: **"Being Prepared"**

- Tracking map: **Shows restaurant location**
- Notification: **"Your biryani is being cooked!"**
- ETA timer: **Adjusts accordingly**

In your GRC app

When someone flags a new risk, the following must all update together:

- Risk count
- Severity summary
- Dashboard widget

Just like Swiggy keeps every screen aligned with the same live order status, your app should keep every related UI element aligned with the same data.

2. To Share Data Between Multiple Parts of the App

1. Many components need the same data.

Example:

The Aadhaar Card Analogy

Your Aadhaar number is needed in many places:

- Bank account opening
- Mobile SIM registration
- Gas subsidy linking
- Passport application
- PAN card linking

Without central data

Different systems may store different versions of your details:

- Bank has your address as **Mumbai**
- Gas agency has **Pune**
- Passport office has **Delhi**
- Mobile company has **Bangalore**



You then spend months correcting mismatches everywhere.

With UIDAI's central database

Your Aadhaar profile becomes **one source of truth**.

Update your address once, and every service pulling Aadhaar data gets the correct version.

In your app

A logged-in user's information such as name, role, and permissions is needed in:

- Navbar: **"Welcome, Ramesh!"**
- Profile page: **Full details**
- Dashboard: **Role-based widgets**
- Settings page: **Edit options based on role**

State management ensures all these components read from **one central user object**.

A logged-in user's name may be needed in the navbar, profile page, dashboard, and settings page.

Without state management, each part may try to manage the same data separately, causing mismatch.

3. To Handle Change Over Time

Applications are **dynamic**. Data is not fixed.

Example:

The IRCTC Tatkal Booking Analogy

Anyone who has booked a Tatkal ticket knows how stressful the journey can be:

```
10:00 AM → Page Loading
10:01 AM → Captcha Appeared
10:02 AM → Payment Processing
10:02 AM → Payment Failed
10:03 AM → Retry Payment
10:04 AM → SUCCESS! Ticket Confirmed
```

Without state management

- You do not know whether payment went through
- The page may show **"Processing"** forever
- You may accidentally book twice
- This can lead to chaos and double charges

With proper state management

The app always knows the exact stage:

- **LOADING** → Show spinner
- **CAPTCHA_REQUIRED** → Show captcha input
- **PAYMENT_PENDING** → Show "Do not refresh" message
- **PAYMENT_FAILED** → Show retry button
- **SUCCESS** → Show ticket with PNR

The application moves through states in a controlled and predictable manner.

1. Loading → Success → Error → Retry → Updated Result

State management helps the app know **what stage it is in**.

4. To Make Behavior Predictable

A good app should react in a clear and controlled way.

Example:

The UPI Payment Analogy (Google Pay / PhonePe)

You are sending ₹5000 to a friend for splitting a trip expense.

Without predictable state

- Google Pay shows **"Payment Successful"**
- Your bank SMS says **"Payment Failed"**
- Your friend says **"I did not receive anything"**
- Your balance shows money deducted

- PhonePe history shows **"Pending"**

This creates total confusion. Did the payment happen or not?

With predictable state management

Either → SUCCESS

- Green checkmark everywhere
- Balance updated
- Friend receives notification
- History shows "Completed"

Or

- FAILED
- Red cross everywhere
- Balance unchanged
- Retry button shown
- History shows "Failed"

Every part of the app reflects the **same final status**. There are no contradictions.

In your GRC app

If a risk submission fails:

- The form should show **"Failed"**
- The notification should say **"Failed"**
- The risk list should **not** show the new risk

There should never be success on one page and error on another.

When a risk submission fails, the app should show a failure status, not success on one page and error on another.

5. To Reduce Bugs as the Application Grows

1. A small app may work with simple local variables.
2. A larger app with many pages, users, and API calls becomes hard to control without structured state management.

The Kirana Shop vs Big Bazaar / DMart Analogy

Small Kirana Shop (Small App)

A small shop can run on memory and informal tracking:

- “Gupta ji ka ₹500 baaki hai”
- “Last week Verma ji bought 2kg rice”
- Works fine with 50 regular customers
- No formal system needed

Big Bazaar / DMart (Large Application)

A large retail chain handles:

- 10,000+ customers daily
- Multiple billing counters
- Inventory across warehouses
- Online orders and in-store purchases
- Discounts, offers, loyalty points
- GST calculations, returns, and exchanges

Can a manager remember all of that mentally? Impossible.

They need:

- Central inventory management
- Unified billing software
- Customer database
- Real-time stock updates

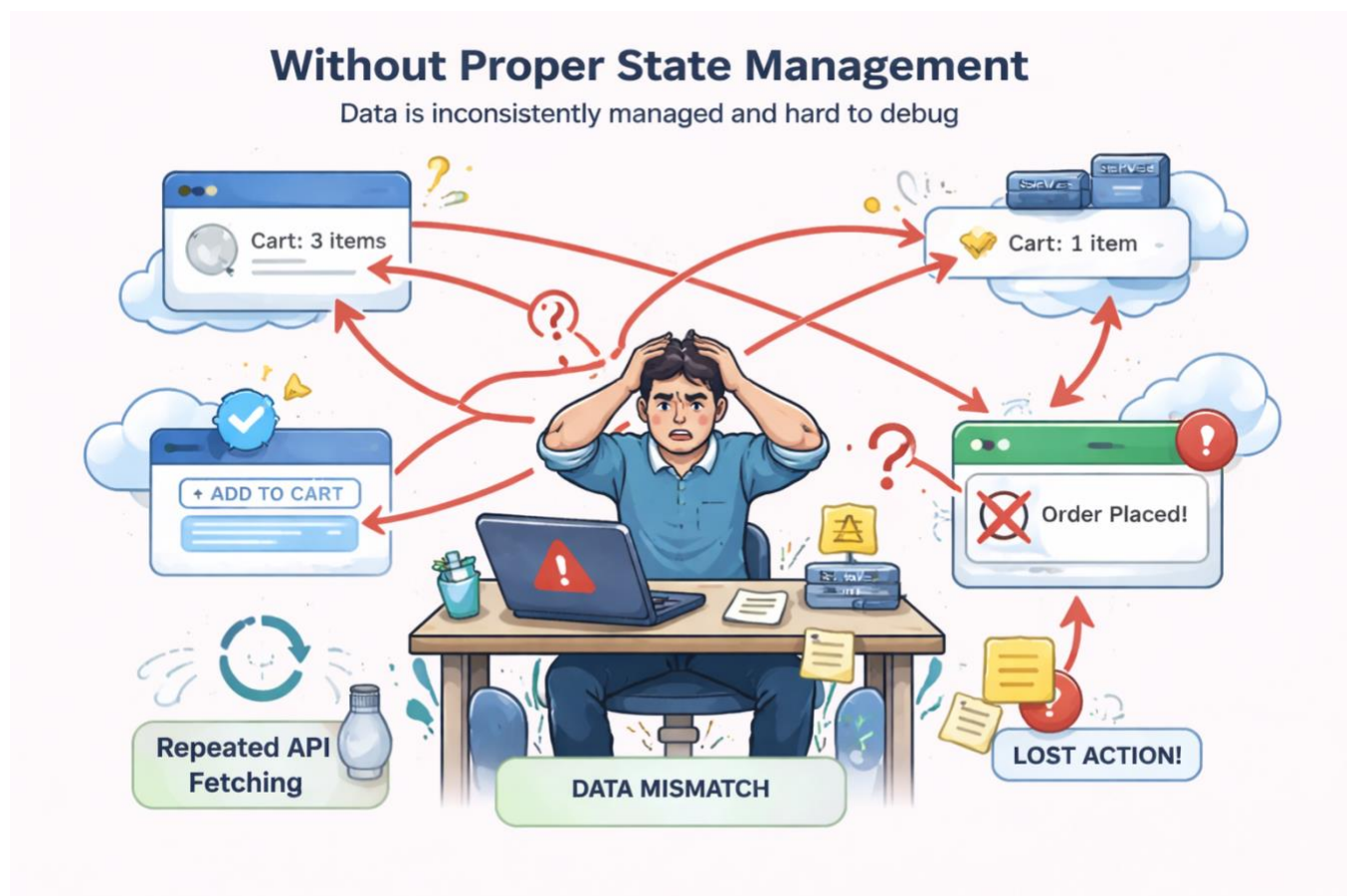
Without structured systems

- Item shows **"In Stock"** online, but the shelf is empty
- Two counters sell the last item to different customers
- Loyalty points do not match
- Returns create accounting confusion

What Happens if State Management Does Not Exist

If state management does not exist, the app may still work for very small situations, but it becomes unreliable as complexity increases.

The major problems are:



1. UI Inconsistency

Different parts of the app show different versions of the truth.

Example:

1. Risk count shows 3 items on the navbar, but the Risk Dashboard shows 2.
-

2. Lost User Actions

The user clicks or types something, but the app forgets it.

Example:

1. Form data disappears when switching tabs.
-

3. Repeated Fetching and Unnecessary Processing

Each component may fetch the same data separately because nothing is centrally tracked.

Example:

1. Profile, sidebar, and dashboard all call the same user API again and again.
-

4. Hard Debugging

When data changes from many places without control, it becomes difficult to know:

1. what changed
 2. when it changed
 3. why it changed
 4. which screen caused the issue
-

5. Poor Scalability

A project without proper state handling may work in the beginning, but later becomes messy and fragile.

6. Race Conditions and Stale Data

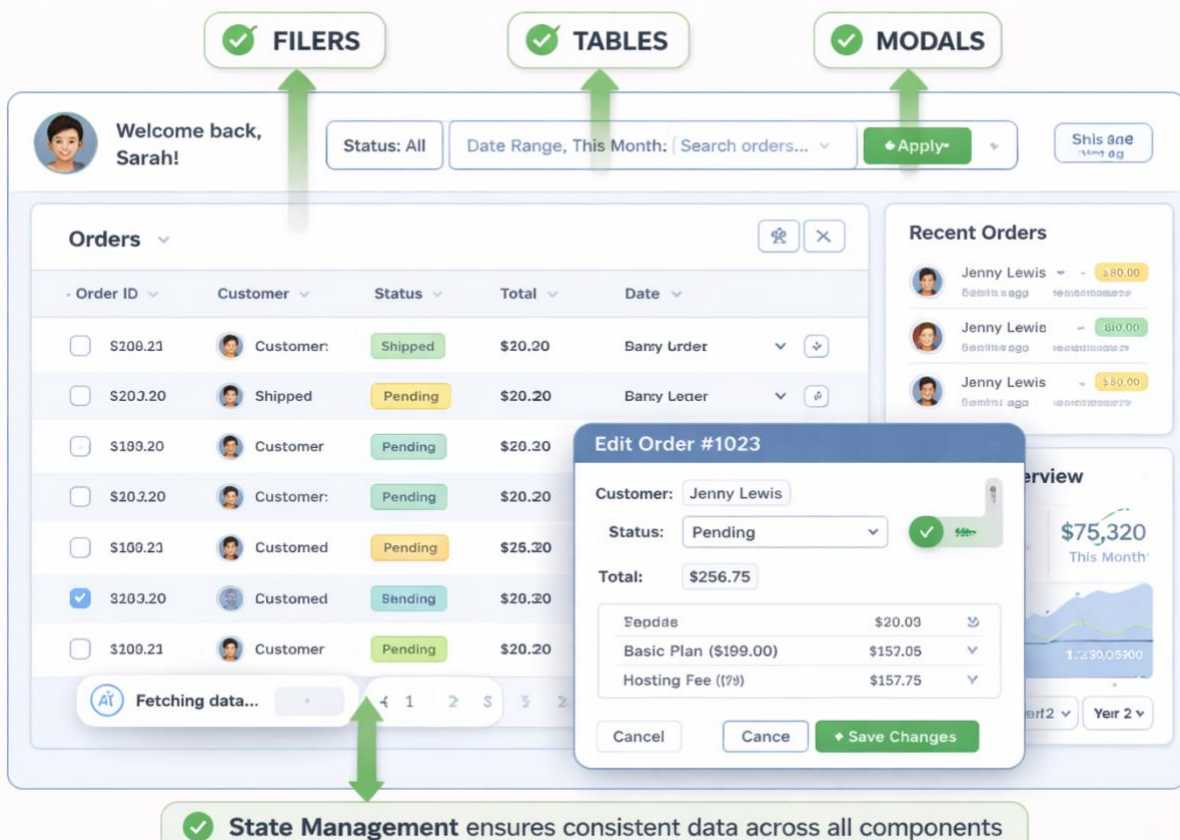
One request finishes later than another and overwrites newer data with older data.

Example:

1. User searches “apple,” then quickly searches “banana.”
2. If “apple” response comes late and replaces “banana,” the UI shows wrong results.

Example Scenarios Where State Management Is Crucial

Scenario 1: Dashboard with Filters, Tables, Modals, User Info, API Data



1. Multiple components need the same data.
2. API responses can arrive at different times.
3. Users can trigger several actions quickly.
4. One part of the UI affects another part.

What Happens if No Proper State Management Exists?

1. Duplicated data logic
2. Filter values mismatch
3. Modal shows old data
4. One section updates, another does not
5. Unnecessary re-renders and API calls

The app starts feeling **unstable**.

Scenario 2: RiskaVaire GRC, RiskaVaire TPRM, CRM, Banking Platform



This app handles:

1. Logged-in sessions

2. Role-based screens
3. Shared business data
4. Approval workflows
5. Notifications
6. Real-time updates
7. Async operations
8. Backend synchronization

What Happens if No State Management Exists?

1. Users may see wrong permissions
2. Stale records may appear
3. Actions may duplicate
4. Workflow status may not match backend reality
5. Audit/history screens may show inconsistent values
6. Debugging becomes expensive and slow

At this stage, lack of state management becomes a **product reliability issue**.

Scenario 3: Chat App, Stock Dashboard, Live Order Tracker, Shared Document Editor

1. Data changes continuously and from multiple sources:
 - i. User input
 - ii. Server push
 - iii. Websockets
 - iv. Background refresh
 - v. Other users' actions

What Happens if No State Management Exists?

1. Unread counts become wrong
 2. Chat order gets mixed
 3. Duplicate messages appear
 4. One user sees updates, another does not
 5. "Online/Offline" states become unreliable
-

Conclusion

State management is required because:

1. Applications must remember and react to changing data consistently.
2. It ensures that the UI and backend stay in sync, and that data flows predictably across different components.

If state management does not exist:

1. The application becomes unreliable and unstable.
2. It leads to bugs, poor performance, and harder maintenance as complexity grows.

Thus, **state management is essential** for building scalable, reliable, and predictable applications.

What Exactly Is State?

State is any data your application remembers and uses to decide what to show. It is the answer to the question: "what is true right now?" Here are concrete examples:

- Is the user logged in? (affects what pages are accessible)
- How many open risks are there? (affects the risk count badge and dashboard totals)
- Which page is currently displayed? (affects routing)
- Are there unread notifications? (affects the badge number)
- Is dark mode enabled? (affects the entire visual theme)
- What search results came back from the API? (affects what the user sees)

The Three Types of State

Not all state is the same. Before you reach for a global store, you need to classify the state you are dealing with. Putting everything into a global store is as bad as having no store at all. This is where most beginners go wrong.

Type	Definition & Examples	Where It Belongs
UI / Local State	Affects only one component. Is this dropdown open? Is this tooltip visible? What is typed in this input field?	Component-level only. React: useState. Vue: ref or reactive. Do NOT put in global store.
Shared / Global State	Needed by multiple components across the app. Logged-in user, active risk filters,	Global store - Pinia (Vue) or Redux Toolkit (React).

State Management · Enterprise Frontend Teaching Resource

	theme preference, notification count, selected organisation.	This is exactly what stores are designed for.
Server / Remote State	Data that lives on the server and must stay synchronised. Risk register, vendor assessments, control frameworks, audit findings, compliance records.	TanStack Query or Vue Query - these handle caching, background refetch, and staleness automatically.



The Three Types of State

Not all state is the same. Before you reach for a global store, you need to classify the state you are dealing with. Putting everything into a global store is as bad as having no store at all. This is where most beginners go wrong.

Type	Definition & Examples	Where It Belongs
 UI / Local State Affects only one component.	Affects only one component. ✓ Is this dropdown open? ✓ Is this tooltip visible? ✓ What is typed in this input field?	Component-level only.  React: useState  Vue: ref or reactive  Do NOT put in global store.
 Shared / Global State Needed by multiple components across the app.	Needed by multiple components across the app. ✓ Logged-in user ✓ Active risk filters ✓ Theme preference ✓ Notification count ✓ Selected organisation	Global store – Pinia (Vue) or Redux Toolkit (React).  Pinia  Redux Toolkit This is exactly what stores are designed for.
 Server / Remote State Data that lives on the server and must stay synchronised.	Data that lives on the server and must stay synchronised. ✓ Risk register ✓ Vendor assessments ✓ Control frameworks ✓ Audit findings ✓ Compliance records	TanStack Query or Vue Query – these handle caching, background refetch, and staleness automatically.  TanStack Query  Vue Query

 **Choose the right type of state for the right job.** Better structure. Fewer bugs. Easier to scale. 

The Golden Rule

Not everything belongs in the global store. A tooltip's visibility, a tab index, a form input value - these are local UI state. Stuffing them into a global store creates noise, makes debugging painful, and causes unnecessary re-renders across your entire app.

Asking the Right Question: A Decision Tree

When you encounter a new piece of data, ask yourself these questions in order:

Step 1

Does this data need to be shared across multiple components?

→ Yes: Continue to Step 2.

→ No: Use local component state and stop here.

Step 2

Does this data come from a server / API?

→ Yes: Use TanStack Query or Vue Query. It handles caching and syncing for you.

→ No: Continue to Step 3.

Step 3

Is this a user preference, session data, or application-level setting?

→ Yes: Use Pinia (Vue) or Redux (React).

→ No: Reconsider - it is probably local state.

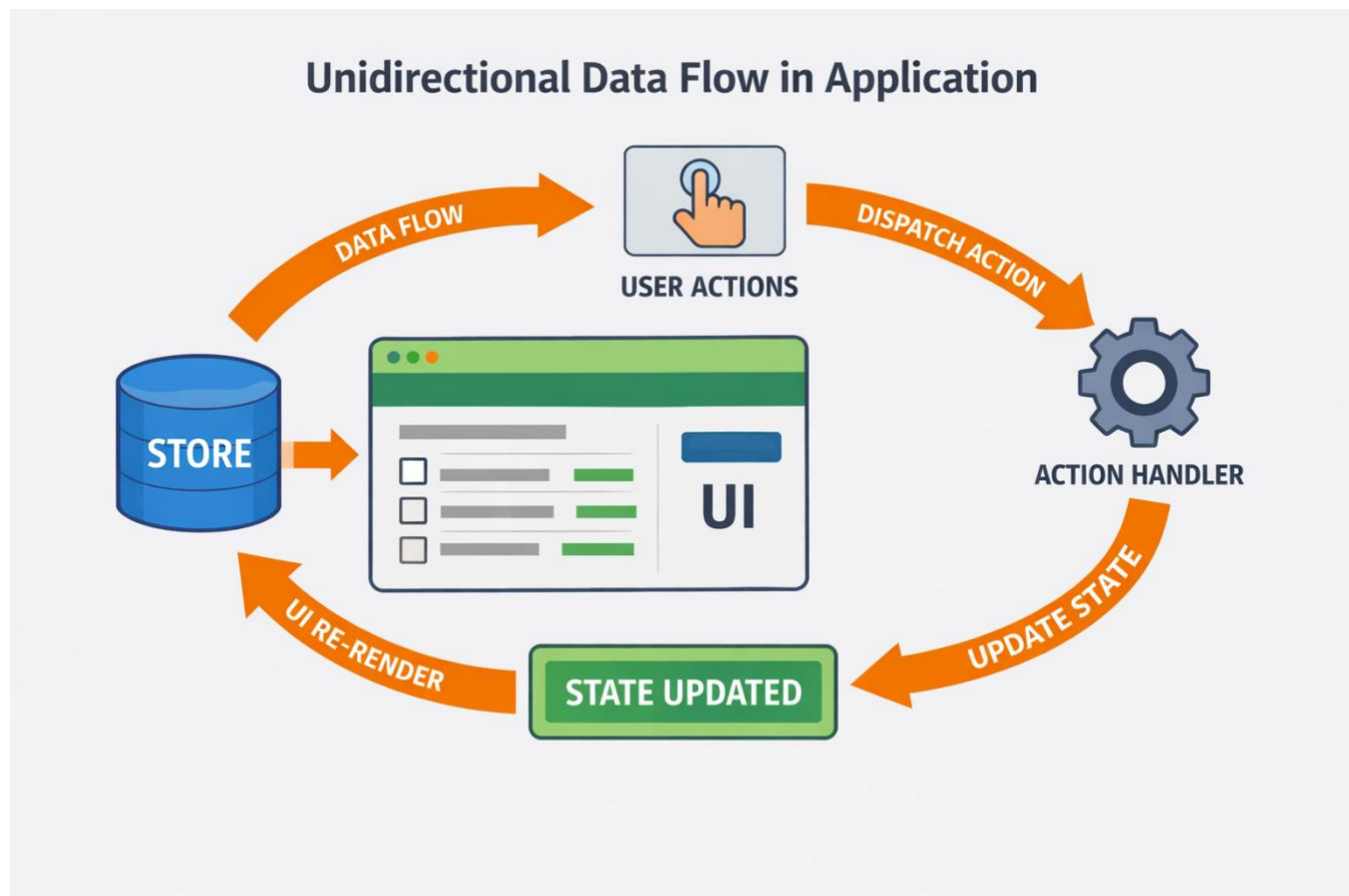
SECTION 02

Data Flow in Applications

Understanding how information moves - the foundational mental model

The Newspaper Analogy

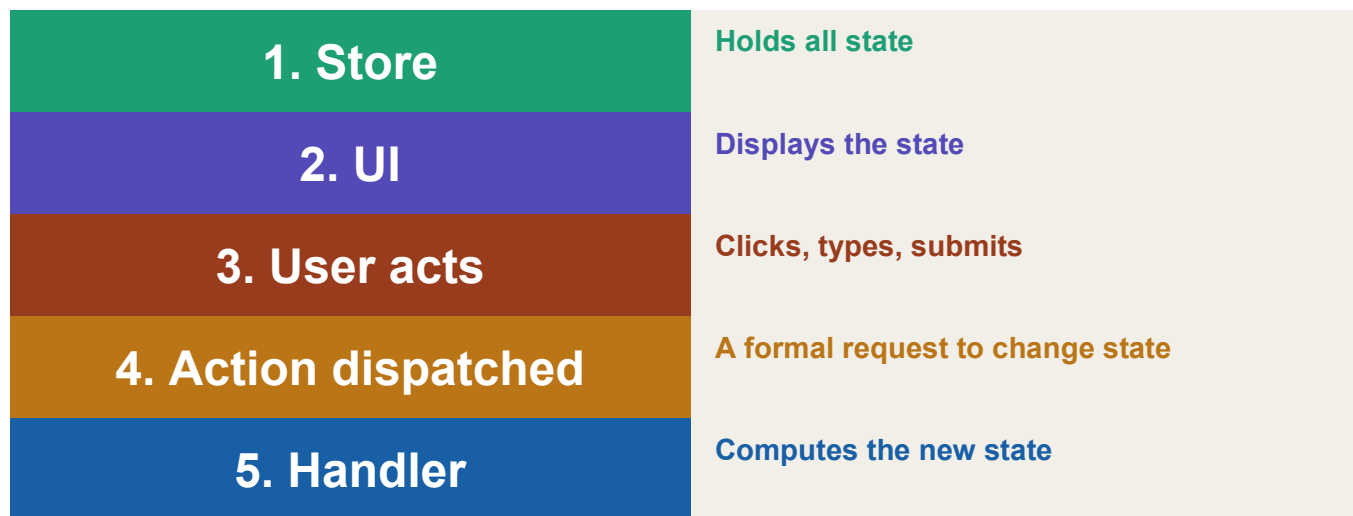
Think of how a newspaper works. A journalist writes a story. The editor approves it. The printing press produces the paper. Every single reader gets the same version. Information travels in one direction only. There is no situation where a reader edits the newspaper and those changes appear for everyone else. That would be chaos.



This one-directional flow is the core principle behind modern state management. In an app, data flows from the store outward to the UI. User interactions travel back inward through a controlled channel (actions/dispatch). Never directly. Never chaotically.

The Universal State Management Loop

Every modern state management system - regardless of framework - follows the same fundamental cycle. Understanding this loop means understanding all of them.



Key Insight

The cycle is: Store → UI displays state → User does something → Action dispatched → Handler computes new state → Store updates → UI re-renders. Every single interaction in your app follows this loop. Its predictability is what makes large-scale apps maintainable.

Why Unidirectional Flow Matters

Before modern frameworks, jQuery-era apps let anything modify anything anywhere. A button click in one component would directly update a variable in another, which would update the DOM somewhere else. The result was untraceable spaghetti - you could not reproduce bugs, you could not undo changes, and you had no audit trail.

Without Unidirectional Flow

✗ Different UI areas show conflicting values

With Unidirectional Flow

✓ Single source of truth for all data

- ✗ Impossible to debug - no trail to follow
- ✗ Cannot reproduce bugs reliably
- ✗ Cannot undo or replay actions
- ✗ State changes are invisible and unpredictable

- ✓ Every change is logged and traceable
- ✓ Can replay history and reproduce any bug
- ✓ Time-travel debugging possible in DevTools
- ✓ Predictable, auditable, maintainable

Real-World Example: The Instagram Like Button

Every time you tap the Like button on Instagram, a very specific sequence plays out. Understanding this sequence is understanding how every modern app works.

User taps the Like button	The component detects the interaction and initiates a change request.
Action is dispatched	A formal request is sent: "Post 123 has been liked." This is the only way state can change.
Handler processes the action	The reducer or action handler receives the request and calculates the new state: liked = true, count += 1.
Store updates	The central store now holds the new state. All parts of the app that read this post are notified.
UI re-renders	The heart icon turns red. The counter updates to show the new count. Instantly.
API call happens in background	The network request to the server happens after the UI has already updated. Users never wait.

This sequence is identical whether it is a status toggle, a risk submission, a form save, or any other user interaction. The pattern does not change. Only the specifics of the action and the resulting state change.

SECTION 03

Pinia - State Management for Vue.js

The official Vue state library. Simple, powerful, and TypeScript-first.

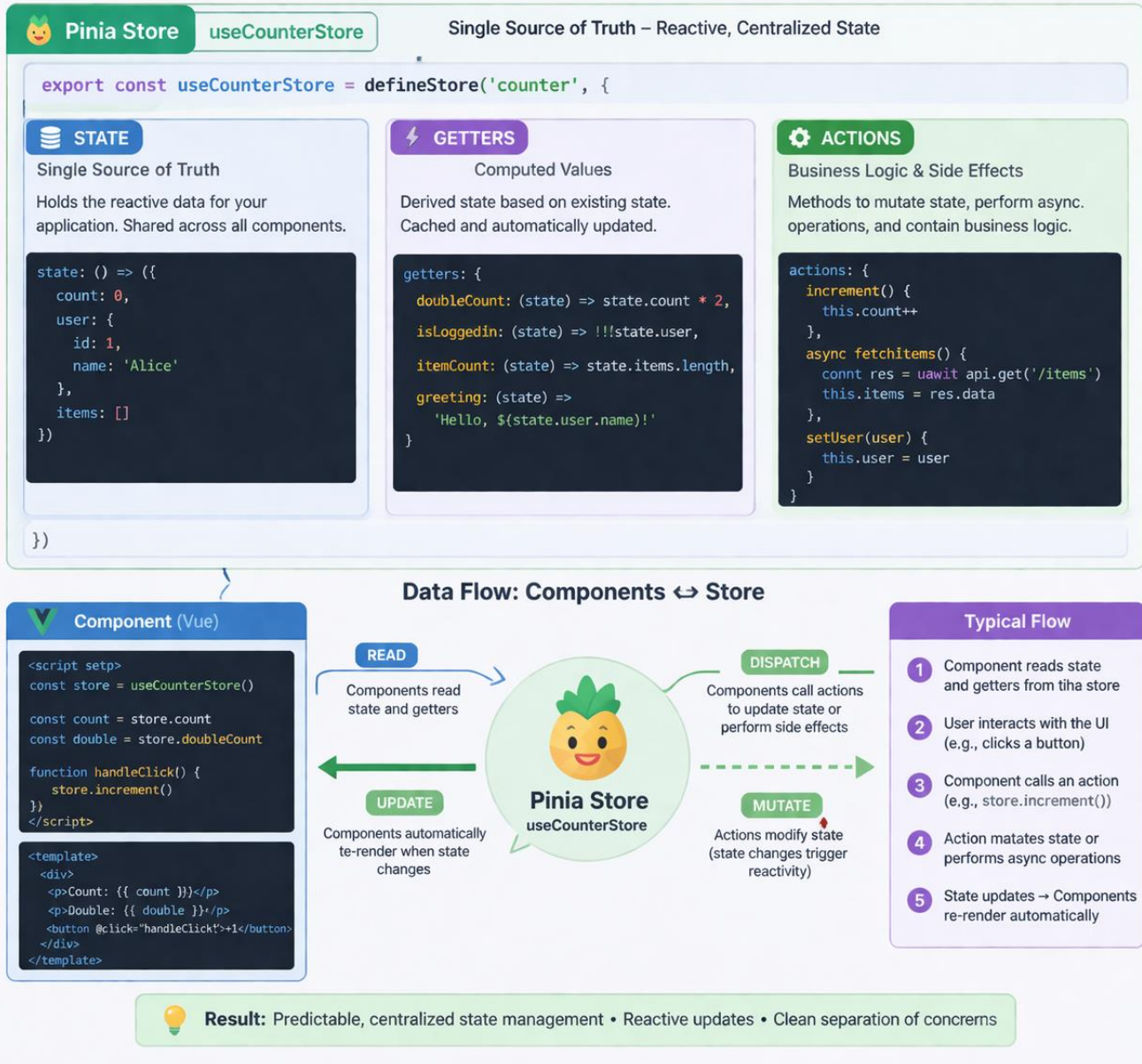
Why We Need Pinia

Imagine a large GRC platform built in Vue (like RiskaVaire GRC). The Navbar needs to show how many open risks exist. The Risk Dashboard needs the full risk register. A Vendor Card needs to know whether to show "Assess Vendor" or "Assessment In Progress." All three components need the same risk data.



Pinia Store – State, Getters, and Actions

Centralized state management for Vue.js applications with a clear data flow



Without Pinia, you would need to pass that risk data up and down through parent and child components - a practice called prop drilling. Every component in between, even if it does not use the risk data, must accept it as a prop and pass it down to its children. This is brittle, tedious, and a maintenance nightmare.

Pinia is like a company's internal intranet portal. Any employee (component), no matter which department (page) they work in, can log in and directly access the company directory (state), view summary dashboards (getters), and submit update requests (actions). No one has to pass information through every floor of the building.

The Problem: Prop Drilling

Without Pinia, your component tree looks like this: the App passes riskCount to Layout, which passes it to Header, which passes it to RiskBadge. None of the intermediate components use the data - they just relay it. Adding any new piece of shared data means updating every level. This is the prop drilling problem. Or worse, every time riskCount is needed an API is called to retrieve the same data.

With Pinia

Every component that needs risk data calls useRiskStore() directly. The Navbar, Risk Dashboard, and Vendor Card all get their data from the same store, without any of them needing to know about the others. The component tree becomes clean and each component is self-sufficient.

The Three Parts of Every Pinia Store

Pinia organises each domain of your application into a dedicated store file. Every store has exactly three parts, and understanding each one is essential.

State - The Raw Data

State is the actual data your store holds. It is reactive - when it changes, every component reading it automatically updates.

Think of it as the database of your store. For a risk store in RiskaVaire GRC, state would contain: risks (the array of risk records), isLoading (whether a network request is in flight), and lastFetched (when the data was last retrieved from the server).

Rule: State should only contain raw data, never values that can be derived from other data.

Getters - Computed Summaries

Getters are computed values derived from state. They are the equivalent of computed properties in Vue components. They are cached - they only recalculate when the state they depend on changes.

For a risk store: totalRisks would count all risks. criticalCount would count high-severity items. isStale would check whether the data is older than 5 minutes.

Rule: Never store a value in state if it can be computed from other state. Always use a getter instead. This prevents inconsistency bugs.

Actions - Operations That Change State

Actions are methods that modify state. They can be synchronous or asynchronous. This is one of Pinia's biggest advantages over older tools - you can call an API directly inside an action without any middleware.

For a risk store: `addRisk()` would add a new risk. `fetchRisks()` would call the API and update state with the response. `clearRisks()` would reset everything.

Rule: State must only ever be modified inside actions. Never modify state directly from a component.

How Components Use the Store

Using a Pinia store from any component is straightforward. You call `useRiskStore()` to get access to the store instance. From that instance you can read any state property or getter, and call any action.

Critical: Use `storeToRefs` for Reactive Destructuring

When you destructure state or getters from a Pinia store in Vue, you must use `storeToRefs()` to preserve reactivity. If you destructure directly without `storeToRefs`, you get plain values that will not update when the store changes. Actions, however, do not need `storeToRefs` - call them directly on the store instance.

The Cache-First Pattern in Pinia

A well-designed Pinia store action does not blindly fetch from the API every time it is called. Instead, it checks whether it already has fresh data before making a network request. This is the cache-first pattern, and it is built into your action logic.

1

Check if cache exists

Does the store already have data? If yes, proceed to Step 2.

2

Check if cache is fresh

Compare the `lastFetched` timestamp against your TTL (e.g. 5 minutes). If fresh, return the cached data immediately - no API call.

3

Fetch from the server

Only if the cache is empty or stale. Mark `isFetching` as `true` so other parts of the app know a request is in progress.

4

Update the cache

Store the fresh data and update `lastFetched`. Any component reading the store automatically gets the new data.

Multiple Stores - One Per Domain

Pinia encourages creating separate stores for each domain: a user store for authentication, a risks store for the risk register, a vendors store for TPRM, a notifications store for alerts. Each store owns its own data, its own logic, and its own actions. Components can use as many stores as they need - just call each store's use function.

SECTION 04

Redux - State Management for React

The Parliament of state managers. Formal, auditable, and powerful.

Why We Need Redux

React's own philosophy is that data flows downward through components via props. This is elegant for small apps. But in a large enterprise application with dozens of deeply nested components, you end up passing the same piece of data through 8 levels of components that do not even use it - they are just passing it along. This is prop drilling, and at scale, it is a maintenance catastrophe.

Redux solves this by making state globally accessible. Any component, no matter how deeply nested, can reach into the Redux store and read exactly the data it needs. And any component can dispatch an action to update that state, without needing to coordinate with parent components.

Redux works like a country's legal system. You cannot walk into Parliament and change a law yourself. You must submit a formal bill (action). It goes through the committee (reducer). If approved, the law books (state) are updated. Every citizen (component) is then governed by the updated law. The process is formal, documented, and auditable - and that is a feature, not a limitation.

Redux Toolkit - The Modern Way

The original Redux required enormous amounts of boilerplate: separate files for action type constants, action creator functions, and reducer functions. Redux Toolkit (RTK) is the Redux team's own solution to this problem. It wraps Redux in a more ergonomic API that eliminates the boilerplate while preserving all the power.

Always Use Redux Toolkit

Legacy Redux (without RTK) is what you will encounter in older codebases. Understand it so you can read it, but never write new code with it. All new Redux code should use Redux Toolkit. It is the official recommendation from the Redux team.

The Slice: Redux's Core Unit

Redux Toolkit introduces the concept of a "slice" - a self-contained module for one feature of your application. A slice bundles together three things that used to be spread across multiple files: the initial state, the action types, and the reducer logic. One slice = one feature = one file.

Initial State

Every slice begins with an `initialState` object that defines the shape and starting values of that slice's data.

For a risks slice: `items` (the array of risk records), `status` ('idle' | 'loading' | 'succeeded' | 'failed'), `error` (any error message), and `lastFetched` (the timestamp of the last successful fetch).

Rule: Always define a status field as a string enum, never as a boolean `isLoading`. This gives you more granular control over the loading lifecycle.

Reducers - Pure Functions for State Changes

A reducer is a pure function: given the same input, it always produces the same output, with zero side effects. It receives the current state and an action, and returns the new state.

RTK uses a library called Immer internally, which means you can write code that looks like direct mutation but is actually creating a safe immutable copy under the hood.

Rule: Reducers handle synchronous, simple state changes. They must never call APIs, trigger other actions, or produce side effects.

`createAsyncThunk` - For Async Operations

API calls and other async operations cannot go inside reducers. Redux Toolkit provides `createAsyncThunk` for this purpose.

An async thunk defines what to do asynchronously (call the API), and automatically generates three action types: pending (request started), fulfilled (request succeeded), and rejected (request failed). You handle these three generated actions in extraReducers within your slice, setting the appropriate status for each.

Selectors - Reading State Efficiently

A selector is a function that extracts a specific piece of state from the Redux store. Rather than accessing state directly in components, you define reusable selector functions and use them with `useSelector()`.

Why Selectors Matter

Selectors serve multiple purposes. They are reusable - write the logic once, use it in ten components. They are memoizable - using `createSelector` from RTK, a selector only recomputes its result when its inputs change, preventing unnecessary re-renders. They are testable - a selector is just a pure function. And they keep components clean - components just call a selector, they do not need to understand the store's shape.

How Components Use Redux

Redux provides two hooks for use in components. `useSelector` takes a selector function and returns the selected state. It subscribes to the store and causes the component to re-render only when the selected value changes. `useDispatch` returns the dispatch function, which you use to send actions to the store.

The Pattern

Read: `const risks = useSelector(selectRiskItems)` - the component now automatically updates whenever risk records change.

Write: `const dispatch = useDispatch()`, then `dispatch(addRisk(riskRecord))` - a formal action is sent through the system.

Async: `dispatch(fetchRisks())` - the thunk starts, sets status to 'loading', makes the API call, then sets status to 'succeeded' or 'failed'.

The Redux DevTools Advantage

One of the most powerful features of Redux is its DevTools integration. Because all state changes flow through reducers - which are pure functions - Redux maintains a complete, auditable history of every action that has ever been dispatched.

-
- See every action dispatched, in order, with its exact payload
 - Click on any past action to see what the state looked like at that moment - this is time-travel debugging
 - Replay a sequence of actions to reproduce a bug exactly
 - Export the entire state history and share it with a colleague
 - Import a state history to load a specific application state for testing

This audit trail is why large enterprise teams prefer Redux for complex state. When a bug is reported, you can export the DevTools log, import it, and see exactly what happened - every click, every API response, every state transition.

SECTION 05

Pinia vs Redux - Key Differences

Same problem, different philosophy. Know when to reach for each.

The Fundamental Philosophical Difference

Both Pinia and Redux solve the same problem: shared state management. But they approach it from fundamentally different philosophical starting points. Understanding this difference will help you use each tool correctly and understand why they are designed the way they are.

Pinia (Vue.js)

Philosophy: Reactive, direct mutation

State is live and reactive - when it changes, Vue's reactivity system automatically propagates those changes to every component that reads it. Actions can mutate state directly, which feels natural and requires less ceremony.

Analogy: A shared office whiteboard. Any authorised person can walk up and update it directly. Everyone in the office can see the change immediately.

Redux Toolkit (React)

Philosophy: Immutable, pure functions

State is never modified in place. Reducers always produce a brand new state object. This immutability gives you a complete, accurate history of every state transition - which enables time-travel debugging and predictable behaviour.

Analogy: A formal amendment process. You must submit the right paperwork (action) for any change to the record (state). Every submission is logged and traceable.

Feature-by-Feature Comparison

Feature	Pinia (Vue)	Redux Toolkit (React)
Boilerplate	Very low - one file, three sections	Low with RTK - slices bundle everything
Mutations	Direct assignment inside actions	Reducers (with Immer under the hood)
Async operations	Native async/await in actions	createAsyncThunk wrapper
Derived values	Getters (like computed properties)	Selectors (via createSelector)
Calling actions	store.actionName()	dispatch(actionName())
TypeScript	Auto-inferred from state definition	Strong support, some manual types
DevTools	Vue DevTools (Pinia tab)	Redux DevTools browser extension

Immutability	Not enforced (reactive system handles it)	Enforced via Immer (write mutably, stored immutably)
Multiple stores	Encouraged - one per domain	One root store with feature slices
Learning curve	Gentle - familiar to Vue developers	Moderate - more concepts to learn

When to Choose Which

In most enterprise projects, this decision is made for you by the technology stack. Vue projects use Pinia. React projects use Redux Toolkit. However, understanding the reasons behind this pairing makes you a better developer.

Choose Pinia if:

- You are building a Vue 3 application - it is the official recommendation and integrates seamlessly.
- Your team values minimal boilerplate and wants to move quickly.
- You want TypeScript type inference to work automatically without extra setup.
- Your state logic is straightforward and does not require complex traceability.

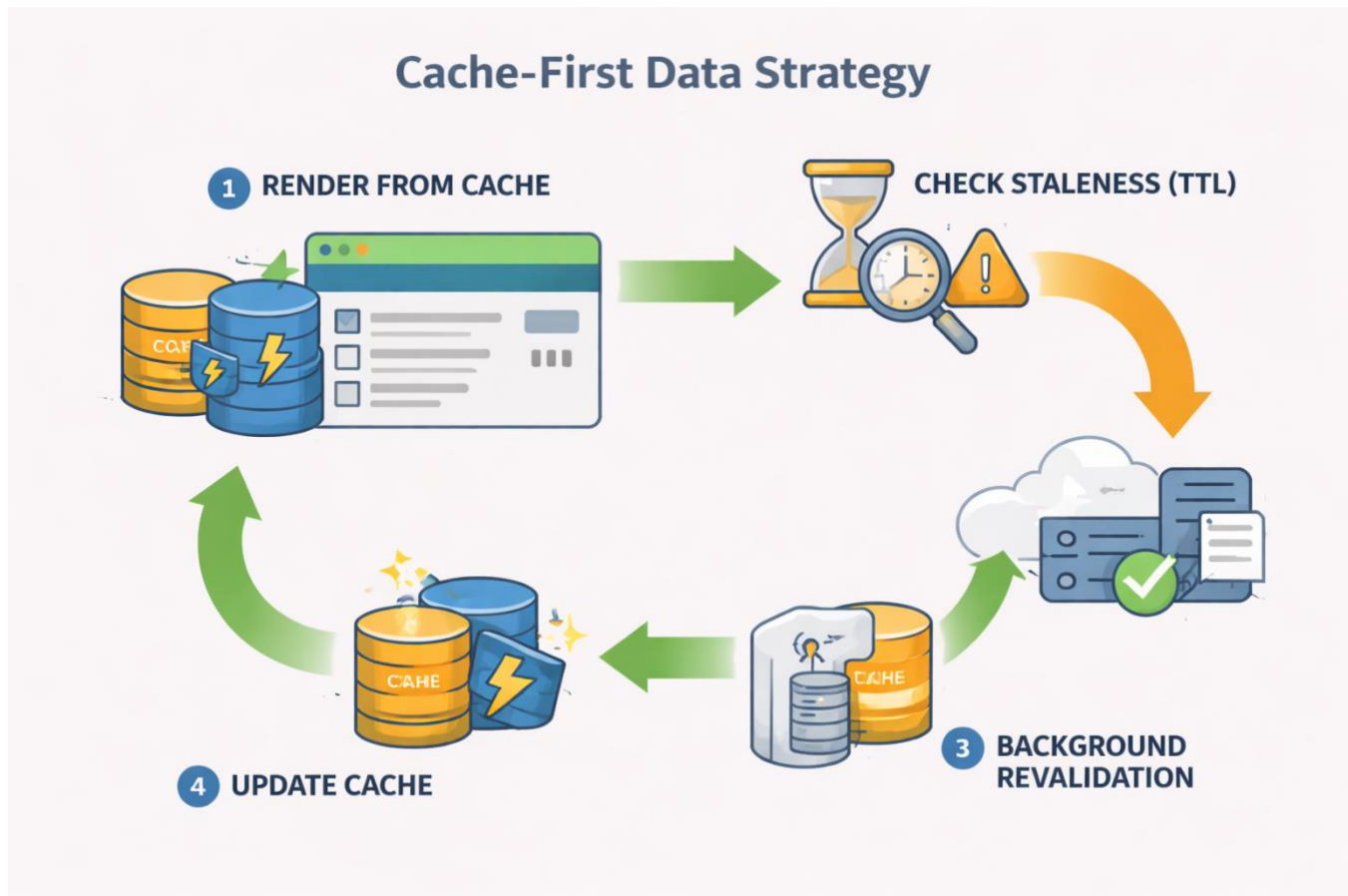
Choose Redux Toolkit if:

- You are building a React application - it is the most widely adopted solution with the largest ecosystem.
- You need strict immutability guarantees and a complete audit trail of all state changes.
- Your team requires powerful time-travel debugging via DevTools.
- You are working on a very large enterprise app with many developers where explicit, traceable state changes reduce errors.

SECTION 06

Cache-First Data Strategy

Make your app feel instant. Never make users wait for data they already have.



The Banking App Analogy

Imagine opening your banking app. In the pessimistic (bad) scenario, you see a loading spinner for three seconds every single time you open it, before your balance appears. In the optimistic (good) scenario, you see yesterday's balance the instant the app opens. A subtle indicator in the corner shows the app is checking for updates. A moment later, the balance refreshes if it changed.

Both scenarios performed the same network request. The only difference is when the UI was shown. The second approach makes the app feel dramatically faster because the user is never blocked from seeing their content.

Think of a newspaper subscription. In the morning, you already have yesterday's newspaper. You read it immediately - you are not blocked waiting for today's edition to be printed. Meanwhile, today's edition is being prepared in the background. When it arrives, you swap it in seamlessly. You were never blocked from reading. You just got an update when it was ready.

Why This Matters in Enterprise Apps

In an enterprise application, users navigate between pages dozens of times a day. Without caching, every navigation triggers a fresh API call, every API call shows a spinner, and every spinner is a moment of perceived slowness. At scale, this adds up to a poor user experience.

The Numbers

Consider a page that takes 300ms to load from the API. Without caching, a user who visits this page 20 times a day waits a cumulative 6 seconds. With cache-first, all but the first visit is instant. The user's perception of the app's speed changes dramatically - and this is achievable with no infrastructure changes, just smart state management.

The Four-Step Cache-First Pattern

The pattern has four steps that execute in order every time a component requests data. Learn these steps - they apply whether you are implementing them manually or relying on TanStack Query to do it for you.

Step 1

Render from cache immediately

When the component mounts, check the store. If data already exists from a previous session or navigation, render it instantly. The user sees content before any network request has been made. If no cached data exists, show a skeleton screen.

Step 2

Check staleness (TTL)

Compare the lastFetched timestamp against your Time-To-Live (TTL). A typical TTL is 5 minutes. If the cached data is younger than the TTL, it is considered fresh - stop here. No API call is needed. The user already has the most current data you can reasonably provide.

Step 3

Background revalidation

If the data is stale, fire an API call in the background. Critically, do not show a loading spinner during this step - the user already has data on screen. The network request is invisible to them. They continue using the app uninterrupted.

**Step
4****Patch only what changed**

When the fresh data arrives, compare it to the cached data. Update only the records that actually changed. Do not replace the entire dataset - this causes unnecessary re-renders and potentially jarring visual changes. Surgical updates produce smooth UI transitions.

Staleness and TTL - How Fresh Is Fresh Enough?

The TTL (Time To Live) determines how long cached data is considered fresh before a background revalidation is triggered. The right TTL depends on how frequently your data changes and how important freshness is to the user experience.

Data Type	Recommended TTL	Reasoning
User profile	15 minutes	Changes infrequently; user expects it to be accurate but not real-time
Risk register	5 minutes	Risk statuses and severities can change; moderate freshness required
Notifications	60 seconds	Users expect near-real-time notification updates
Stock price / live data	0 seconds	Always fetch fresh; caching is inappropriate for real-time data
Search results	2 minutes	Acceptable to show slightly stale results; dramatically speeds up repeat searches
Vendor assessment list	30 seconds	Must reflect latest TPRM data; frequent refresh needed

TanStack Query - The Enterprise Solution

Implementing the full cache-first pattern manually requires careful engineering. TanStack Query (for React) and Vue Query (for Vue) automate all four steps and handle a long list of edge cases you would otherwise need to manage yourself.

- Automatic caching with configurable TTL via the `staleTime` option
- Background refetching when data becomes stale
- Deduplication of requests - if ten components request the same data simultaneously, only one network call is made
- Retry on failure with exponential backoff - failed requests automatically retry up to three times

- Refetch on window focus - when the user switches back to your app after being away, data is revalidated
- Refetch on network reconnect - when connectivity is restored, stale data is refreshed automatically
- Built-in DevTools for inspecting the cache, query status, and stale state

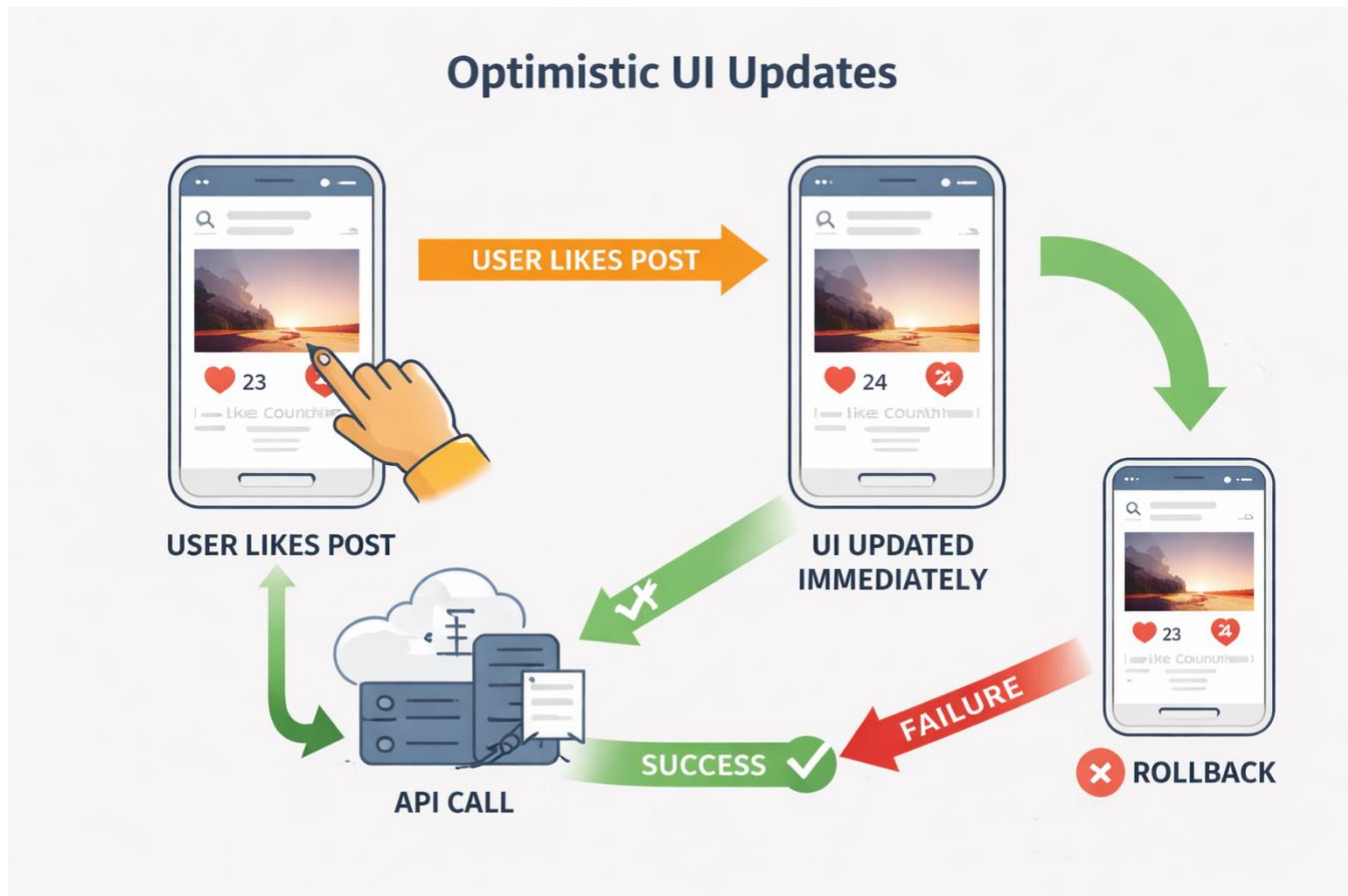
Recommended Architecture

At scale, the best pattern is: use Pinia or Redux for client-side global state (user session, UI preferences, application settings) and TanStack/Vue Query for all server state (anything fetched from an API). This separation of concerns means each library does what it is best at.

SECTION 07

Optimistic UI Updates

Assume success. Correct for failure. Make every action feel instant.



The WhatsApp Analogy

When you send a message in WhatsApp, your message appears in the chat the instant you press Send. You do not wait for a server response. You do not see a loading spinner. The message is simply there. WhatsApp assumes the network request will succeed and shows you the result immediately.

Meanwhile, the network request happens in the background. A single grey tick appears (sent from your device). A double grey tick (delivered to the server). Double blue ticks (read by the recipient). If the request fails, the message gets a red warning icon and offers a retry button. The message is not removed - that would be jarring. It stays in the chat, flagged for your attention.

This is the optimistic UI pattern in its most recognisable form. Update the user interface immediately, assuming the server call will succeed. Handle failure gracefully if it does not. Instagram, Twitter, Google Docs, and every major modern product uses this pattern.

Pessimistic vs Optimistic - Side by Side

The contrast is stark when you compare both approaches for the same interaction.

Pessimistic UI

Perceived speed: Slow

User clicks "Approve Risk"
Button shows spinner and is disabled
User waits 200–2000ms for server
Server responds with success
Button changes to "Approved" ✓
Status updates from Pending to Approved

Optimistic UI

Perceived speed: Instant

User clicks "Approve Risk"
Button IMMEDIATELY changes to ✓ Approved
Status IMMEDIATELY shows Approved
Server call happens in background
Success: nothing changes (already correct)
Failure: rollback to Pending + error toast

The Implementation Pattern - Five Steps

The optimistic update pattern is always the same five steps, regardless of the action being performed.

1

Save a snapshot

Before changing anything, save a copy of the current state for the item you are about to modify. This snapshot is your rollback point if the server call fails.

2

Update the store immediately

Mutate the state as if the server has already confirmed success. The UI renders instantly.

3

Fire the API call

Send the network request to the server in the background. The user is not blocked and can continue interacting with the app.

4

On success - do nothing

If the server confirms success, the UI is already showing the correct state. Nothing needs to change. Optionally, sync any server-generated data (like exact counts or generated IDs).

5**On failure - rollback**

Restore the store from the snapshot you saved in Step 1. The UI reverts to its previous state. Show an error toast or inline message explaining what went wrong and offering a retry.

When to Use Optimistic Updates

This pattern is powerful, but it is not appropriate for every action. The key question is: how costly is a rollback if the server fails? For reversible, low-stakes actions, optimistic updates create a dramatically better experience. For high-stakes or irreversible actions, optimistic updates are dangerous.

Good use cases

- ✓ Risk status toggles (mark as accepted, approved)
- ✓ Bookmarks and favourites (pin a risk, flag a vendor)
- ✓ Toggle switches (notifications, settings)
- ✓ Drag-and-drop reordering
- ✓ Mark as read / unread
- ✓ Inline field edits (risk description, vendor notes)
- ✓ Pinning or unpinning risks and controls

Bad use cases

- X Audit submissions and sign-offs
- X Account or compliance record deletion
- X Sending compliance notifications or alerts
- X Password or security changes
- X Regulatory filings and legal submissions
- X Any action that cannot be undone
- X Anything with serious regulatory or audit consequences

The Principle

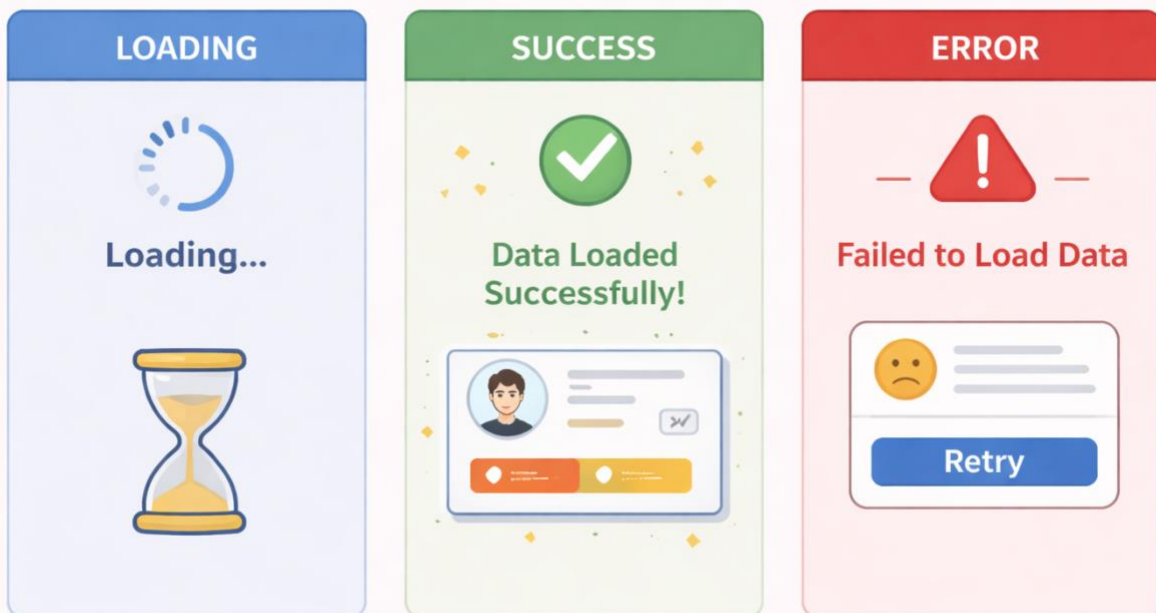
If approximately 90% of this action will succeed on the first try, and rolling back on the 10% failure case is straightforward, use optimistic updates. Why make users wait 100% of the time for a success case that happens 90% of the time?

SECTION 08

UI Best Practices

Building the visual layer on top of well-managed state.

Async Operation States



The Three States of Every Async Operation

Every network request - every API call, form submission, or file upload - has exactly three possible outcomes. Your UI must be designed for all three, every single time. Designing only for success is the most common and most visible mistake in enterprise frontend development.

Loading The request is in flight. Show a skeleton screen that mirrors the	Success	Error Something went wrong. Show a helpful message, explain what
---	----------------	--

real layout. Never a blank white page.

Data arrived. Display the content from the store. Components read state and render it - nothing more.

happened, and always offer a retry button.

The loading state is what your users experience every time they open the app on a slow connection. The error state is what they experience when the server is down. Both deserve exactly as much design care as the success state. Budget time for all three.

Skeleton Screens vs Spinners

When data is loading, you have two main options for giving users feedback: a spinner (a rotating icon indicating "something is happening") or a skeleton screen (placeholder shapes that mirror the real layout). Enterprise applications should always prefer skeleton screens for page-level loading.

Generic Spinner - Avoid

- ✗ Provides zero layout information
- ✗ Users have no idea what is loading
- ✗ Jarring transition when content appears
- ✗ Feels slow - anxiety-inducing
- ✗ Provides no sense of progress

Skeleton Screen - Prefer

- ✓ Shows exact layout structure
- ✓ Users know what is coming
- ✓ Smooth transition to real content
- ✓ Perceived as faster - less anxious
- ✓ Used by Facebook, LinkedIn, YouTube, GitHub

Rule

Use skeleton screens for any initial page or section load. Use a small inline spinner only for contained, user-triggered actions like submitting a single field or uploading a file - where the user initiated the action and knows something specific is happening.

Smart vs Presentational Components

One of the most important architectural patterns in enterprise frontends is the separation of smart (container) components from presentational components. This separation keeps your codebase testable, reusable, and maintainable as it scales.

Think of a TV news broadcast. The news anchor (presentational component) just reads what is on the teleprompter. They do not go out and gather the news - that is someone else's job. The production team (smart component) decides what to show, gathers information from sources, decides the order,

and puts it on the teleprompter. The anchor is reusable - the same anchor can present any news with any script.

Smart / Container Components

Connect directly to the store (Pinia store or Redux useSelector/useDispatch).

Handle all data fetching and async logic.

Decide which state to show - loading, success, or error.

Pass data down to presentational components as props.

Call store actions in response to user events.

Examples: RiskListPage, VendorContainer, ControlsGridContainer.

Presentational Components

Know nothing about the store - they receive only props.

Focus entirely on rendering UI based on what they receive.

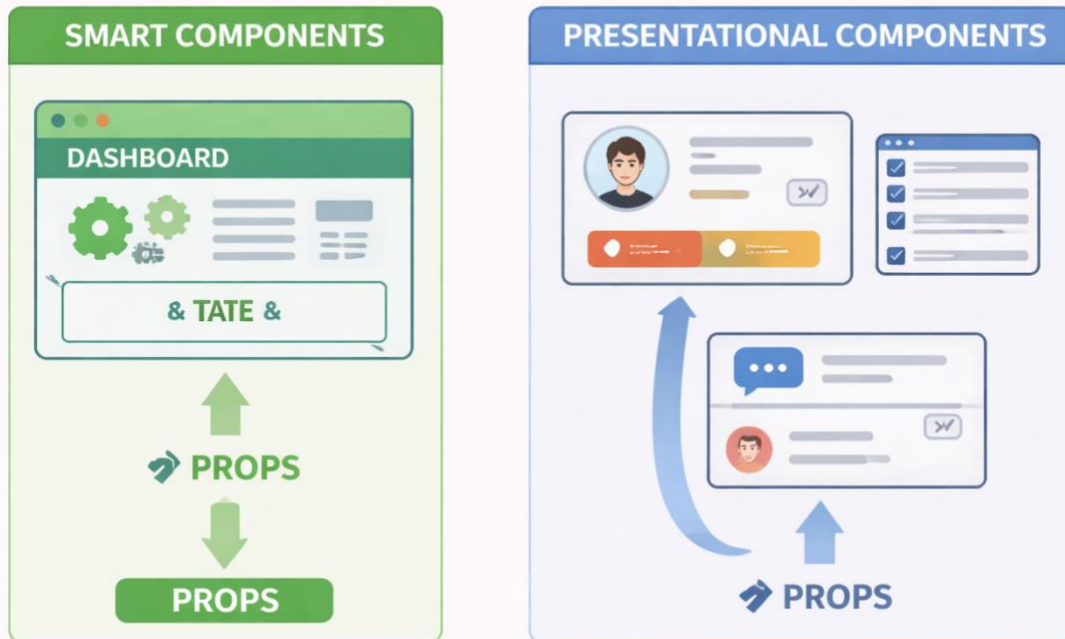
Emit events upward when users interact (clicks, form changes).

Never call APIs directly.

Are fully reusable - the same component works with any data source.

Examples: UserCard, RiskCard, VendorItem, DataTable, Button.

UI Components (Smart vs Presentational)



Memoisation - Preventing Unnecessary Re-renders

As your application grows, re-render performance becomes critical. Every time a parent component re-renders, all of its children re-render too - unless you explicitly prevent it. Memoisation is the practice of caching the result of a function and only recomputing it when its inputs change.

In React

`React.memo` wraps a presentational component and prevents it from re-rendering if its props have not changed. Use it on any component that receives stable props but whose parent re-renders frequently. `useMemo` memoises the result of an expensive computation (like filtering or sorting a large list). Only recomputes when its dependency array changes.

`useCallback` memoises a function reference. Use it when passing callbacks as props to memoised child components - without it, every render creates a new function, breaking the memoisation.

`v-once` renders a component or element only once, ignoring future updates - useful for completely static content.

The diagram illustrates the difference between normalized and unnormalized database states. A large blue arrow points from the 'NORMALIZED STATE' on the left to the 'UNNORMALIZED STATE' on the right.

NORMALIZED STATE

- Users**
 - id: 1, name: Alice
 - id: 2, name: Bob
- Posts**
 - id: 101, title: "Post One", userId: 1
 - id: 102, title: "Post Two", userId: 2

✓ FLAT STRUCTURE
✓ REFERENCES
✓ NO DUPLICATION

UNNORMALIZED STATE

- Users**
 - id: 1, name: Alice
 - posts: {
 - id, title: {Post One'}
 - id: 2, name: Bob
 - posts: {
 - id: 102, {Post Two'}

! DUPLICATED DATA
- SLOW ACCESS
! COMPLEX UPDATES

How you structure data in the store has a major impact on performance and correctness. Deeply nested arrays of objects create duplication, slow lookups, and inconsistency bugs. Normalised state is flat, with entities stored in dictionaries keyed by their IDs.

State Management | Susheel Ramadoss
Confidential

```
risks: [ { id: 1, title: "...", owner: { id: 10, name: "John" }, controls: [ { id: 100, owner: { id: 10, name: "John" } } ] } ]
```

Problem: John's data is duplicated. If John's role changes in RiskaVaire, you must find and update every instance. One missed update creates an inconsistency.

Normalised (recommended)

```
users: { byId: { 10: { id: 10, name: "John" } }, allIds: [10] }
```

```
risks: { byId: { 1: { id: 1, ownerId: 10, controlIds: [100] } }, allIds: [1] }
```

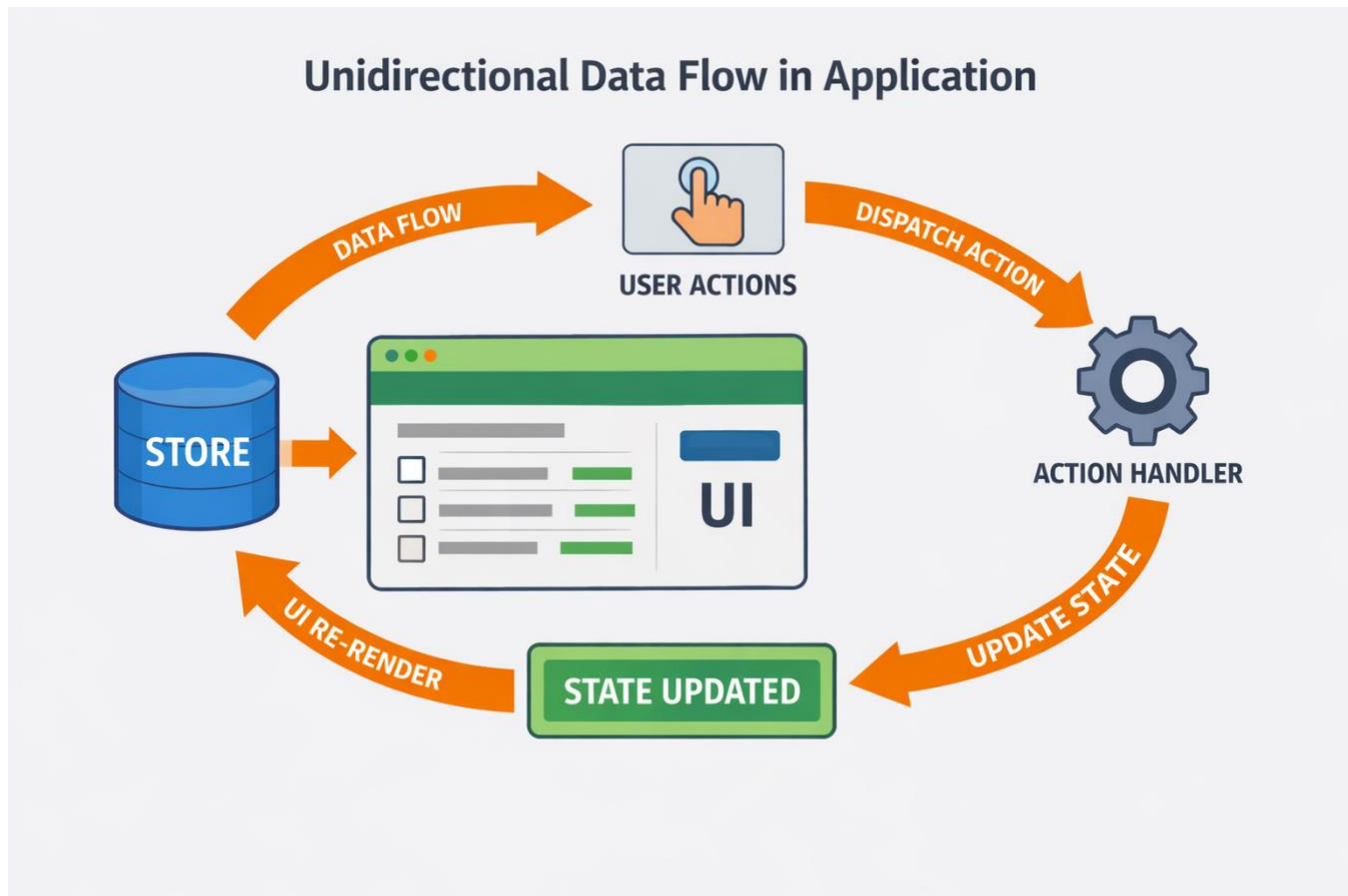
```
controls: { byId: { 100: { id: 100, ownerId: 10, text: "..." } }, allIds: [100] }
```

Benefit: John's data exists in exactly one place. Update his role once in RiskaVaire and it is correct everywhere. Lookups by ID are $O(1)$ - instant, not a loop through an array.

SECTION 09

Full Architecture - How All Layers Connect

Putting it all together: a complete end-to-end flow.



The Complete Stack

A well-architected enterprise frontend application is not a single monolith - it is a set of clearly defined layers, each with a specific responsibility. Understanding these layers and how they communicate is what separates senior frontend engineers from beginners.

UI Layer

Presentational components render data received as props. Smart containers connect to the store and manage loading/error states. Error boundaries catch crashes and show fallbacks. Skeleton screens fill the space while data is loading.

State Management Layer	Pinia (Vue) or Redux Toolkit (React) holds all global client state. Getters and selectors compute derived values. Actions and reducers are the only way state can change. DevTools gives you a complete audit trail.
Data Fetching Layer	TanStack Query or Vue Query manages all server state - caching, background refetching, deduplication, and retry logic. This layer talks to the state management layer when results need to be stored globally.
API Communication Layer	An HTTP client (Axios or native fetch) handles all network requests. Request interceptors add authentication tokens automatically. Response interceptors handle common errors globally (401 Unauthorized, 500 Server Error). Retry logic handles transient failures.
Error Handling Layer	Error boundaries (React) or global error handlers (Vue) catch unhandled exceptions. A toast notification system surfaces errors to users. An error logging service (like Sentry) captures bugs in production.

Tracing a Complete User Interaction

Let us trace exactly what happens from the moment a user flags a new risk in RiskaVaire GRC to the moment all parts of the app reflect the change. This end-to-end trace touches every layer.

1	User submits a new risk	The RiskForm presentational component emits an event. Its smart container component intercepts it.
2	Smart container calls store action	<code>riskStore.addRisk(riskRecord)</code> in Pinia, or <code>dispatch(addRisk(riskRecord))</code> in Redux.
3	Optimistic update fires first	The store immediately adds the risk to the risks array and updates the total. The UI re-renders instantly - the risk count badge shows the new number.
4	API call fires in background	The action sends <code>POST /api/risks</code> to the server. The user is not blocked - they can continue reviewing other risks.
5	Success path	Server confirms the submission. The store syncs any server-generated data (risk ID, assigned owner). Cache timestamp is updated. UI is already showing correct data - no change needed.

6	Failure path	Server rejects the request (network error, validation failure). The store is restored from the snapshot. Risk count reverts. An error toast appears: "Could not submit risk. Please try again."
7	All components auto-update	Because all components read from the same store, the Navbar badge, Risk Dashboard, and Risk Summary widget all reflect the final state automatically. One store update, infinite consumers.

SECTION 10

Recommended Tool Stack

The definitive enterprise frontend toolkit for 2025 and beyond.

For Vue.js Projects

Tool	Purpose	Why This One
Pinia	Global client state management	Official Vue 3 recommendation. Replaces Vuex. Zero boilerplate, full TypeScript support, built-in DevTools.
Vue Query (TanStack)	Server state - all API data	Automates caching, background refetch, retry, and staleness. Eliminates 80% of data-fetching code.
Axios	HTTP client for API calls	Interceptors for auth tokens and error handling. Better than native fetch for enterprise apps.
Vue DevTools	Debugging and state inspection	Inspect Pinia store state in real time, see component trees, profile performance.
VeeValidate + Yup	Form validation	Schema-based validation that integrates cleanly with Vue's reactivity and Pinia.

For React Projects

Tool	Purpose	Why This One
Redux Toolkit	Global client state management	Official Redux recommendation. Eliminates legacy boilerplate. Immer, createSlice, createAsyncThunk included.
TanStack Query	Server state - all API data	The gold standard for React server state. Background refetch, caching, deduplication, and DevTools built in.
Axios	HTTP client for API calls	Request and response interceptors. Automatic JSON parsing. Timeout and progress handling.
Redux DevTools	State debugging	Time-travel debugging, action replay, state diff view. Non-negotiable for Redux projects.
React Hook Form + Zod	Form validation	Minimal re-renders, TypeScript schema validation, excellent performance at scale.

Why TanStack Query Changes Everything

TanStack Query deserves special attention because it fundamentally changes how you think about data fetching. Instead of managing loading states, caching logic, and retry logic yourself in every store action, you declare what data you want and let TanStack Query handle everything else.

- `staleTime` - how long before data is considered stale and a background refetch is triggered
- `cacheTime` - how long unused data stays in memory before being garbage collected
- `refetchOnWindowFocus` - automatically revalidates when the user returns to the browser tab
- `refetchOnReconnect` - revalidates when the device's network connection is restored
- `retry` - number of times to retry a failed request before giving up (default: 3)
- optimistic updates - built-in `onMutate`, `onError`, and `onSettled` callbacks for the full optimistic pattern

Architecture Recommendation

Use TanStack Query for all server state (risk registers, vendor assessments, control frameworks - anything from an API). Use Pinia or Redux for client state (authentication session, theme preference, UI settings, active filter selections). Each library does what it is designed for.

SECTION 11

The 10 Golden Rules

Print these out. Put them on the wall. Enforce them in code review.

These ten rules represent the condensed wisdom of large-scale frontend development. They are not stylistic preferences - they are architectural decisions with measurable consequences. Every rule here exists because its violation has caused real, costly bugs in real applications.

Rule 1**Not everything belongs in the global store**

Tooltip visibility, tab indices, form input values, accordion states - these are local UI state. Keep them inside the component. The global store is for data shared across multiple disconnected components. Over-storing creates noise, causes unnecessary re-renders, and makes DevTools useless.

Rule 2**Server state and client state are different problems**

Data from an API (risks, vendors, controls, audit findings) is server state - use TanStack Query or Vue Query, which handle caching and freshness automatically. Preferences, session data, and UI settings are client state - use Pinia or Redux. Using a global store for API data means rebuilding what TanStack Query already does, poorly.

Rule 3**Always design for all three async states**

Every network request has loading, success, and error outcomes. Every UI that triggers an API call must handle all three. Shipping code that only handles success is an unfinished feature. The loading state is experienced by every user on every page load. The error state is experienced whenever the server is down.

Rule 4**Use skeleton screens for page-level loading**

A skeleton screen shows the layout structure and dramatically reduces perceived load time. A generic spinner provides zero information and creates anxiety. The only exception: use an inline spinner for a specific, user-triggered action on a small component where the layout is already established.

Rule 5**Use optimistic updates for low-stakes reversible actions**

Likes, bookmarks, toggles, drag-and-drop reordering - update the UI immediately, fire the API call in background, roll back gracefully on failure. Never use this pattern for payments, deletions, or security-sensitive actions. Always save a snapshot before mutating, so rollback is always possible.

**Rule
6****Show cached data immediately, validate in background**

Never block the user on a network request when you already have data in the store. Show it. Fire the API call silently. Update only what changed when the response arrives. The user's perception of your app's speed depends on this pattern more than any other optimisation.

**Rule
7****Separate smart and presentational components**

Smart components connect to stores, fetch data, and manage state. Presentational components receive props and render UI. This separation makes components independently testable, reusable across different data sources, and easier to reason about. Never put a store call inside a presentational component.

**Rule
8****Normalise all list data in the store**

Store entities as flat dictionaries keyed by ID, not as nested arrays. `users.byId[10]` instead of searching `users.find(u => u.id === 10)`. This makes lookups instant, updates surgical (change one place, correct everywhere), and eliminates data duplication bugs. Deeply nested state is a design smell.

**Rule
9****Never store derived values - compute them**

If a value can be calculated from other state, it must be a getter or selector, not a stored property. Storing derived values creates the risk of inconsistency - the raw data and the derived value can drift out of sync. `totalPrice` should always be computed from `items`, never stored separately.

**Rule
10****Reset all stores on logout**

When a user logs out, every store must be cleared to its initial state. If the next user logs in on the same device, they must never see any trace of the previous user's data - even momentarily. This is a security requirement. Build a `resetAll` utility that clears every store, and call it on every logout path.

SECTION QR

Quick Reference

The essential summary - one page to rule them all.

The Five Core Principles

Single source of truth	Every piece of data has exactly one authoritative location. No duplication. All components read from the same store.
Unidirectional data flow	Data flows one way: Store → UI → Action → Reducer/Handler → Store. Nothing bypasses this cycle. Nothing mutates state directly.
Cache first, validate async	Show what you have. Check freshness. Update in background. Never block the user on a network request when you already have data.
Optimistic UI for speed	Assume success. Update immediately. Roll back on failure. Make every user interaction feel instant, regardless of network conditions.
Design for all three states	Every async operation has loading, success, and error outcomes. Every UI must handle all three. Building only for success is an incomplete feature.

Complete Feature Comparison

Feature	Pinia (Vue)	Redux Toolkit (React)	TanStack Query
Primary purpose	Client-side global state	Client-side global state	Server state caching
Async operations	Native in actions	createAsyncThunk	Built-in (useQuery, useMutation)
Caching	Manual (lastFetched + TTL)	Manual (lastFetched + TTL)	Automatic with staleTime

Background refetch	Manual implementation	Manual implementation	Automatic on window focus
Optimistic updates	Manual (snapshot + rollback)	Manual (snapshot + rollback)	Built-in callbacks
DevTools	Vue DevTools (Pinia tab)	Redux DevTools extension	React Query DevTools
Boilerplate	Very low	Low (with RTK)	Very low
Learning curve	Gentle	Moderate	Gentle
TypeScript	Auto-inferred	Good support	Excellent

Further Learning

- Pinia official documentation: pinia.vuejs.org
- Redux Toolkit documentation: redux-toolkit.js.org
- TanStack Query documentation: tanstack.com/query
- Vue.js official style guide: vuejs.org/style-guide
- Redux DevTools GitHub: github.com/reduxjs/redux-devtools

Your Next Steps

Build a small project applying these patterns. Set up DevTools and observe state flow. Identify one component in your codebase that violates the smart/presentational split and refactor it. Implement cache-first loading for one API call. Add optimistic updates to one user interaction. Share this document with your team.